2026-5-6 C++模板

- 1.泛型编程
- 2.函数模板
  - 2.1函数模板概念
  - 2.2 函数模板的格式
  - 2.3 函数模板的原理
  - 2.4 函数模板的实例化
    - 2.4.1隐式实例化
    - 2.4.2 显示实例化
  - 2.5 模板参数的匹配原则
- 3.类模板
  - 3.1类模板的定义格式
  - 3.2类模板的实例化

1.泛型编程

我们怎么实现一个通用的交换函数呢?
可以用我们之前学过的函数重载来实现，但是函数重载有一些不好的地方：

1. 重载的函数仅仅是类型不同，代码复用率比较低，只要有新类型出现时，用户就需要自己怎加对应的函数。
2. 代码的可维护性比较低，一个出错可能所有的重载均出错。
   那么我们能否告诉编译器一个模子，让编译器根据不同的类型利用该模子来生成代码呢?
   那么我们就引出了泛型编程：编写与类型无关的通用代码，是代码复用的一种手段，模板是泛型编程的基础。

2.函数模板

2.1函数模板概念

函数模板代表了一个函数家族，该函数模板与类型无关，在使用时被参数化，根据实参类型产生函数的特定类型版本。

2.2 函数模板的格式

```
template<typename T1,typename T2,typename T3.....typename Tn>
```

```
返回值类型 函数名（参数列表）{实例化}
```

函数模板实际上可以与函数参数做一些对比：返回值类型 函数名（参数列表）{实例化}
template:关键字，模板的意思
typename：关键字，用于定义模板参数，也可以使用class替代

```
template<typename T>
void Swap(T& left, T& right) {
    T temp = left;
    left = right;
    right = temp;
}
```

2.3 函数模板的原理

函数模板并不是函数本身，是编译器为同一函数功能匹配参数类型的摸具。所以函数模板就是将本来应该我们做的重复的事情交给编译器。
在编译器编译的阶段，编译器会根据不同的实参类型推演出针对不同类型的函数以供调用。

2.4 函数模板的实例化

用不同类型的参数使用函数模板时，称为函数模板的实例化。函数模板的实例化分为两种：隐式实例化和显示实例化。

2.4.1隐式实例化

让编译器根据实参推演模板参数实际类型：

```
template<typename T>
T Add(const T& left, const T& right) {
    return left + right;
}
void text_template01() {
    //1.函数模板的隐式实例化：让编译器根据实参推演出模板参数的实际类型
    int a1 = 1, a2 = 3;
    double b1 = 1.1, b2 = 2.5;
    Add(a1, a2);
    cout << a1 <<" "<<a2 << endl;
    Add(b1, b2);
```

```
    cout << b1 <<" "<<b2 << endl;
    //在下述语句中编译器会通过实参将a1T推演为int,将b1的T推演为double但是我们现在的
    //模板参数列表中只有一个T，编译器无法确定到底将这个T推演为int还是double而报错
    //Add(a1, b1);
    //这个时候有两种解决方案：1.用户自己来强制转化 2. 使用显示实例化
    Add(a1,(int)b1);//我们还可以使用显示实例化
}
```

在这里，我提一个问题，下面的情况可以吗?

```
template<typename T>
void Swap(T& left, T& right) {
    T temp = left;
    left = right;
    right = temp;
}
void text(){
    int a1 = 1, a2 = 3;
    double b1 = 1.1, b2 = 2.5;
    Swap(a1,(int)b1);
    Swap<int> (a1,b1);
}
```

这种情况无论你采用那种解决方案编译都不会通过的：因为`Swap(a1, (int)b1)`中，`(int)b1`是一个 右值（临时对象） 而`Swap`参数是 `T&`（非常量左值引用），C++ 禁止将非常量左值引用绑定到右值。说人话就是b1在强制类型转化的过程中会产生临时对象，临时对象具有常性，传递给`Swap()`函数进行交换后就被销毁了。但示例中`Swap<int>(a1, b1)`会先将`b1`转换为`int`（截断），然后交换的是`a1`和一个临时`int`，原`b1`不变。这个点可以在注释中说明。

2.4.2 显示实例化

在函数名后的<>中指定模板参数的实际类型

```
void text_template02() {
    int a = 10;
    double b = 20.1;
    Add<int>(a, b);
}
```

如果类型不匹配，编译器会尝试进行隐式类型转换，无法转化编译器会报错。

2.5 模板参数的匹配原则

1. 一个非模板函数可以和同名的函数模板同时存在，而且该函数模板还可以被实例化为该非模板函数。

```
//专门处理int的加法函数
int Add(int left, int right) {
    cout << "haha" << endl;
    return left + right;
}
template<class T>
T Add(T left, T right) {
    cout << "xixi" << endl;
    return left + right;
}
void text_template03() {
    Add(1, 2);//与非模板函数匹配，编译器不需要特化
    Add<int>(1, 2);//调用编译器特化的Add版本
}
```

2. 对于非模板函数和同同名的模板函数，如果其他条件都相同，在调用时会优先调用非模板函数而不会再通过模板函数产生一个实例，如果模板可以产生一个具有更好匹配度的函数，则会选择函数模板。

```
int Add(int left, int right) {
    cout << "haha" << endl;
    return left + right;
}
template<class T1,class T2>
T2 Add(T1 left, T2 right) {
    cout << "xixi" << endl;
    return left + right;
}
void text_template04() {
    Add(1, 2);//与非模板函数完全匹配，不需要函数模板实例化
    Add(1, 2.2);//函数模板可以生成更加匹配的版本，编译器根据实参生成更加匹配的Add函数
}
```

3. 模板函数不允许自动类型转换，但是普通类型可以进行自动类型转换

3.类模板

3.1类模板的定义格式

```
template<class T1,class T2,class T3......,class Tn>
class 类模板名{
//类内成员定义,让类内成员可定义
}
```

示例代码如下：

```
#include<iostream>
using namespace std;
//类模板
template<typename T>
class Stack
{
public:
    Stack(size_t capacity = 4) {
        _array = new T[capacity];
        _capacity = capacity;
        _size = 0;
    }
    void Push(const T& data);
    ~Stack() {
    delete[] _array;   // 建议补充析构函数
    }
private:
    T* _array;
    size_t _capacity;
    size_t _size;
};
template<class T>
void Stack<T>::Push(const T& data)
{
    //扩容
    _array[_size] = data;
    ++_size;
}
int main() {
    Stack<int> st1;
    Stack<double> st2;
```

```
    return 0;
}
```

3.2类模板的实例化

类模板的实例化与函数模板的实例化不同，**类模板的实例化需要在类模板名字后面跟<>**,然后将实例化的类型放在<>中即可，**类模板名字不是真正的类，而实例化的结果才是真正的类。**

```
//Stack是类名，Stack<int>才是类型
Stack<int> st1;//int
Stack<double> st2;//double
```

对比维度	函数模板	类模板
定义格式	`template<typename T>` `T func(T param)`	`template<typename T>` `class Stack { ... };`
实例化方式	隐式实例化（自动推导） 显式实例化（手动指定）	**只能显式实例化**（必须指定类型）
实例化写法	`Add(a, b)` 或 `Add<int>(a, b)`	`Stack<int> st;`
默认参数	不支持模板参数默认值	支持（如 `size_t capacity = 4`）
特化	支持全特化	支持全特化与偏特化
成员函数定义	不涉及	可在类外定义，需加 `template<typename T>`
类型推导	编译器可根据实参推导	必须显式指定
典型用途	通用算法（交换、排序、加法等）	通用容器（vector、stack、list 等）